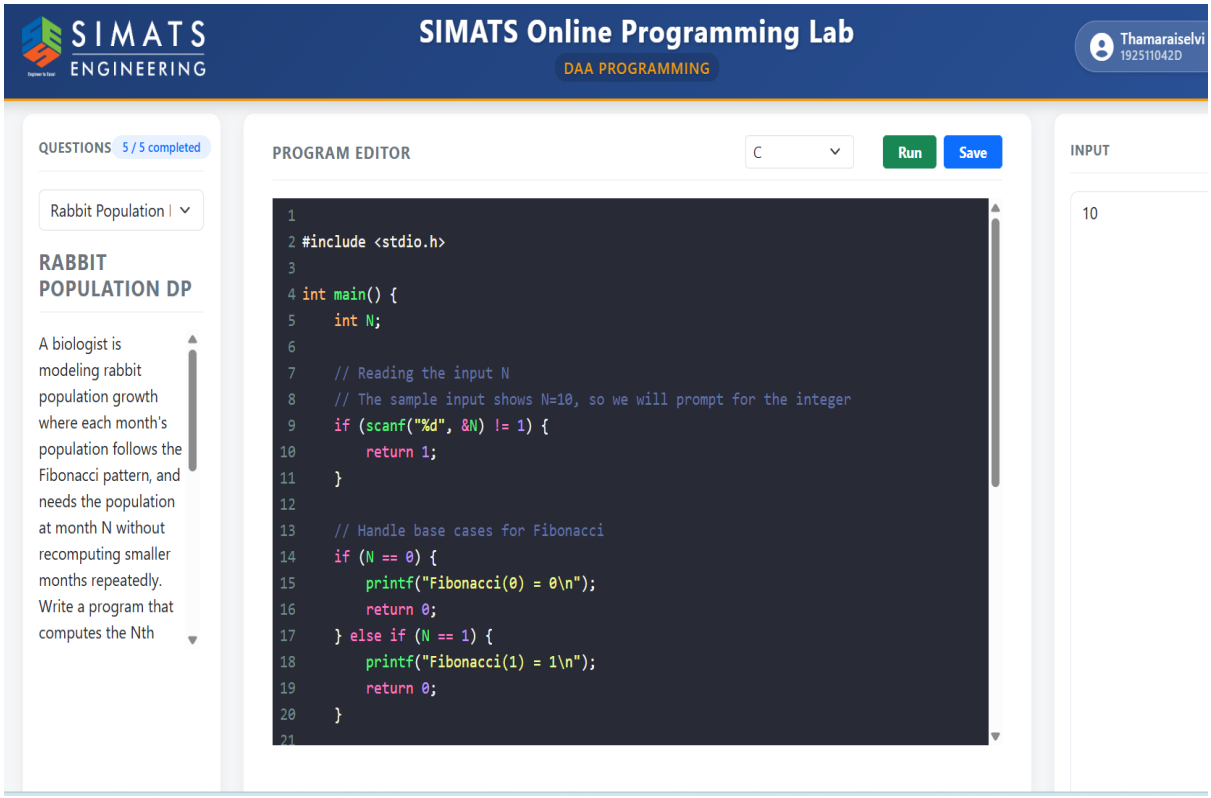
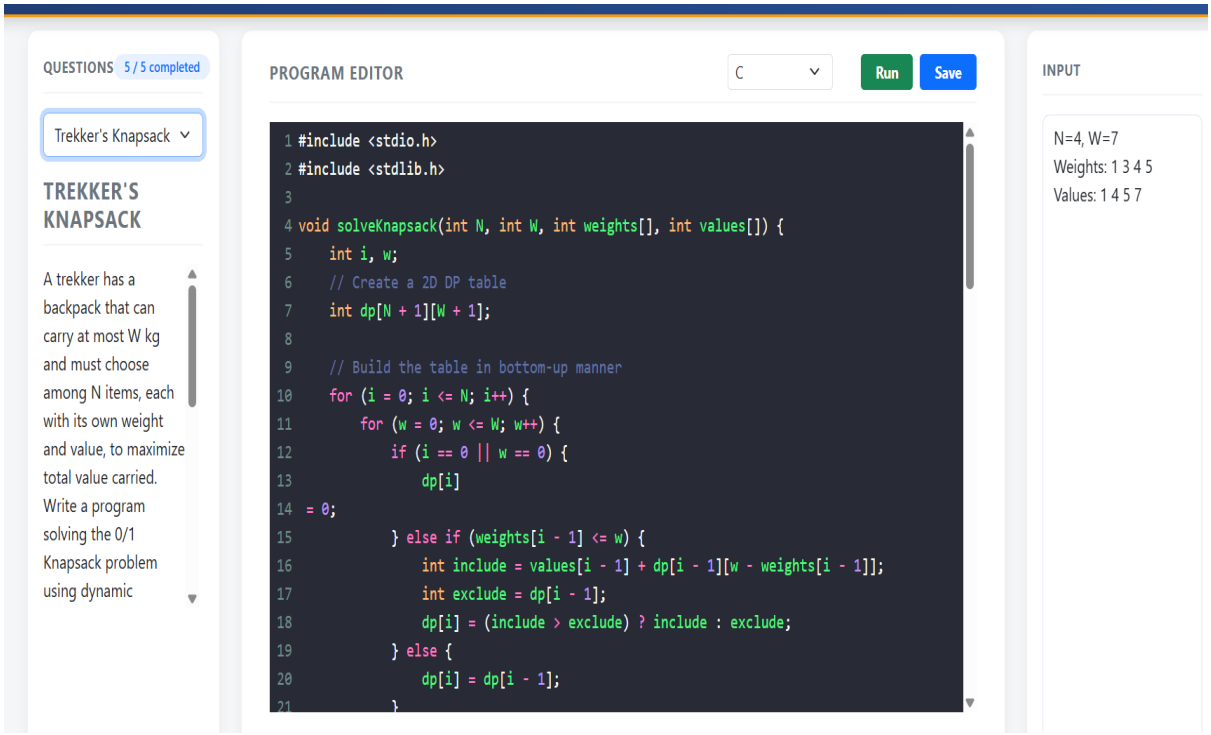
EXP 1:


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Thamaraiselvi
192511042D
QUESTIONS 5 / 5 completed
Rabbit Population I
RABBIT POPULATION DP
A biologist is modeling rabbit population growth where each month's population follows the Fibonacci pattern, and needs the population at month N without recomputing smaller months repeatedly. Write a program that computes the Nth
PROGRAM EDITOR
C
Run
Save
INPUT
10
#include <stdio.h>
int main() {
int N;
// Reading the input N
// The sample input shows N=10, so we will prompt for the integer
if (scanf("%d", &N) != 1) {
return 1;
}
// Handle base cases for Fibonacci
if (N == 0) {
printf("Fibonacci(0) = 0\n");
return 0;
} else if (N == 1) {
printf("Fibonacci(1) = 1\n");
return 0;
}

EXP 2:


QUESTIONS 5 / 5 completed
Trekker's Knapsack
TREKKER'S KNAPSACK
A trekker has a backpack that can carry at most W kg and must choose among N items, each with its own weight and value, to maximize total value carried. Write a program solving the 0/1 Knapsack problem using dynamic
PROGRAM EDITOR
C
Run
Save
INPUT
N=4, W=7
Weights: 1 3 4 5
Values: 1 4 5 7
#include <stdio.h>
#include <stdlib.h>
void solveKnapsack(int N, int W, int weights[], int values[]) {
int i, w;
// Create a 2D DP table
int dp[N + 1][W + 1];
// Build the table in bottom-up manner
for (i = 0; i <= N; i++) {
for (w = 0; w <= W; w++) {
if (i == 0 || w == 0) {
dp[i]
= 0;
} else if (weights[i - 1] <= w) {
int include = values[i - 1] + dp[i - 1][w - weights[i - 1]];
int exclude = dp[i - 1];
dp[i] = (include > exclude) ? include : exclude;
} else {
dp[i] = dp[i - 1];
}

EXP 3:

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Thamaraiselvi S
192511042D

QUESTIONS 5 / 5 completed

Vending Machine C

VENDING MACHINE CHANGE

A vending machine must give change for an amount using the fewest coins possible from the available denominations. Write a program that finds the minimum number of coins needed to make a given amount using dynamic programming.

PROGRAM EDITOR

C

Run Save

```c

#include <stdio.h>
#include <limits.h>

void findMinCoins(int coins[], int n, int amount) {
    int dp[amount + 1];

    // Initialize dp array with a value representing infinity
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        dp[i] = INT_MAX;
    }

    // Fill the dp array using the bottom-up approach
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i) {
                int sub_res = dp[i - coins[j]];
                if (sub_res != INT_MAX && sub_res + 1 < dp[i]) {
                    dp[i] = sub_res + 1;
```

INPUT

```
3
1 3 4
6
```

EXP 4:

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Thamaraiselvi S
192511042D

QUESTIONS 5 / 5 completed

Staircase Ways DP

STAIRCASE WAYS DP

A fitness app counts, for a staircase of N steps, how many distinct ways a person can climb it if they take either 1 or 2 steps at a time. Write a program that computes the number of distinct ways to climb N stairs using dynamic

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main() {
    int n;

    // Reading the input N
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    // Handle base cases for small N
    if (n == 0 || n == 1) {
        printf("Number of ways to climb %d stairs: 1\n", n);
        return 0;
    }

    // dp array to store the number of ways for each step
    long long dp[n + 1];

    // Base cases as specified in the instructions
    dp[0] = 1;
```

INPUT

```
5
```

EXP 5:


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Thamaraiselvi S
192511042D
QUESTIONS 5 / 5 completed
House Robber Plar
HOUSE ROBBER PLANNING
A burglar plans to rob houses along a street, each holding a certain amount of cash, but robbing two adjacent houses in one night triggers a connected alarm, so no two adjacent houses can both be robbed. Write a program that finds the maximum total
PROGRAM EDITOR
C
Run
Save
INPUT
5
2 7 9 3 1
#include <stdio.h>
#include <stdlib.h>
void findRobbedHouses(int n, int cash[], int dp[]) {
int i = n - 1;
int robbed[n];
int count = 0;
while (i >= 0) {
// If we are at the first house, or the current dp value
// comes from robbing this house (dp[i-2] + cash[i])
if (i == 0 || dp[i] == dp[i - 1]) {
i--; // Skip this house, it wasn't robbed
} else {
robbed[count++] = i + 1; // Store house number (1-indexed)
i -= 2; // Move to the house before the previous neighbor
}
}
// Print the houses in ascending order
printf("Houses robbed: ");